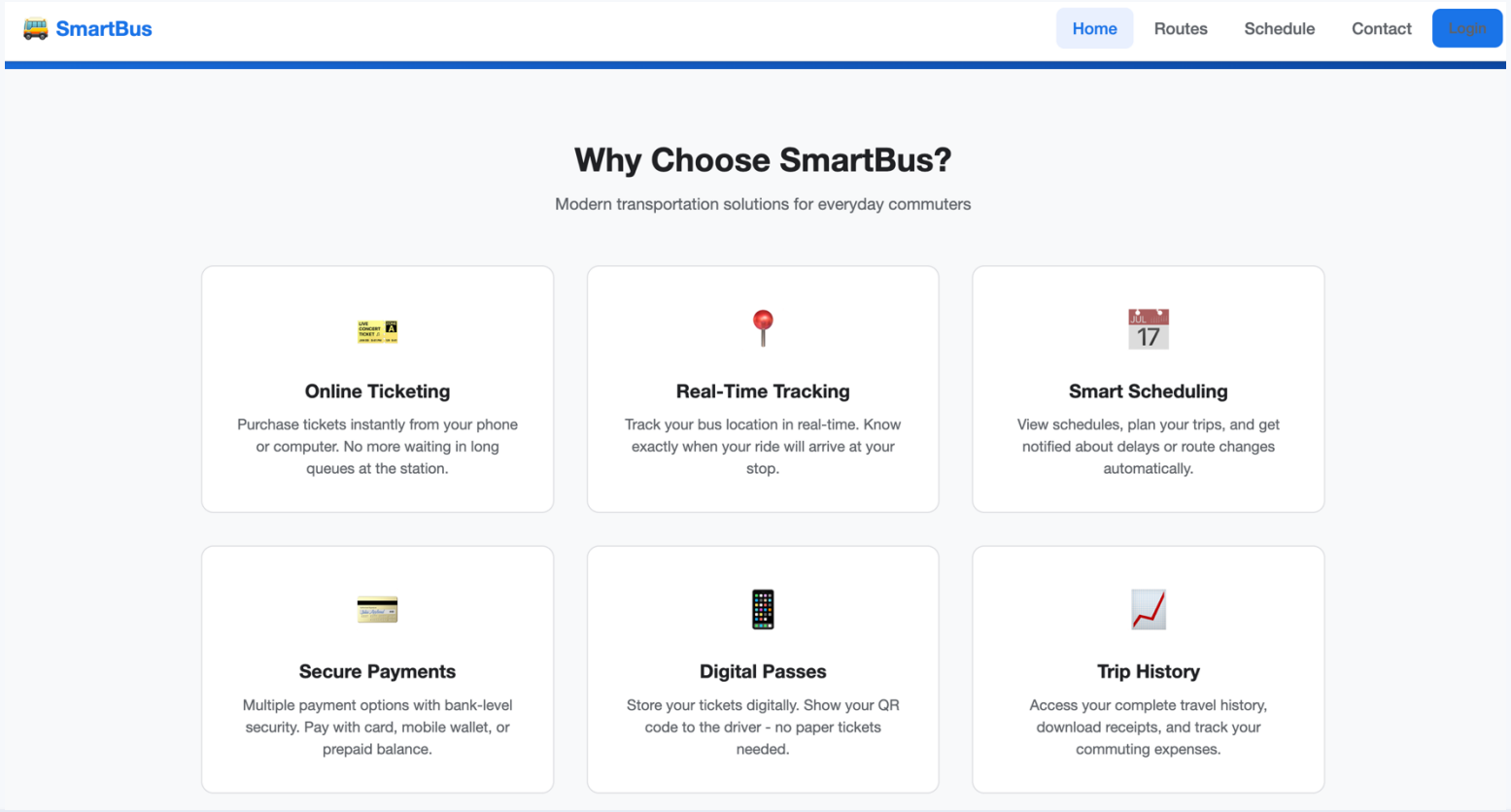
SmartBus

Smart Transportation & Ticketing Integration



Course: CSE 598 — Software Integration & Engineering

Repo: https://github.com/sherzodbeek/smart-bus (frontend prototype)

Presenter: Sherzod Omonov

Problem & Motivation

What commuters struggle with

- Long queues and manual ticketing
- Unclear schedules and route info
- Limited visibility into delays
- Fragmented experiences (tickets, routes, support)

Goal for SmartBus

- One place to browse routes + schedules
- Buy and store tickets digitally (QR)
- User dashboard for trips and tickets
- Admin portal for operations (routes, buses, users)

SmartBus is a UI prototype demonstrating an integrated transportation experience.

It focuses on the end-to-end flow: discover → plan → purchase → manage → administer.

Solution Overview

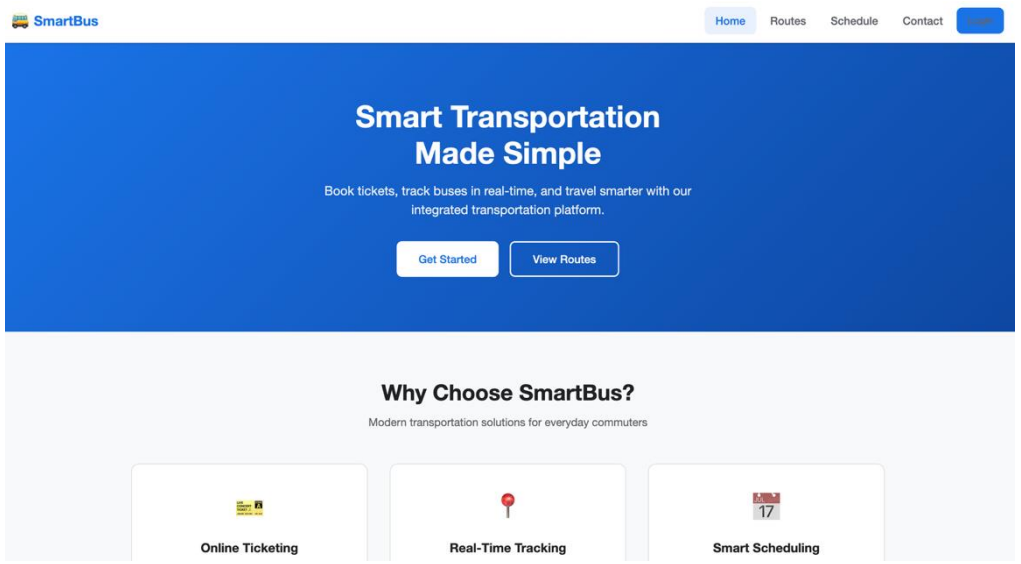
Core value (as implemented in the UI)

- Online ticketing + digital passes (QR concept)
- Browse routes with frequency and fare
- Schedule table with filters (route/day)
- Contact page with FAQ accordion
- Separate User and Admin portals

Two roles:

User Portal | **Admin Portal**

Landing page (mock)



Project / Site Structure

Public Website

- index.html
- routes.html
- schedule.html
- fares.html
- contact.html
- login.html
- register.html

User Portal

- user/dashboard.html
- user/buy-ticket.html
- user/my-tickets.html
- user/routes.html
- user/profile.html

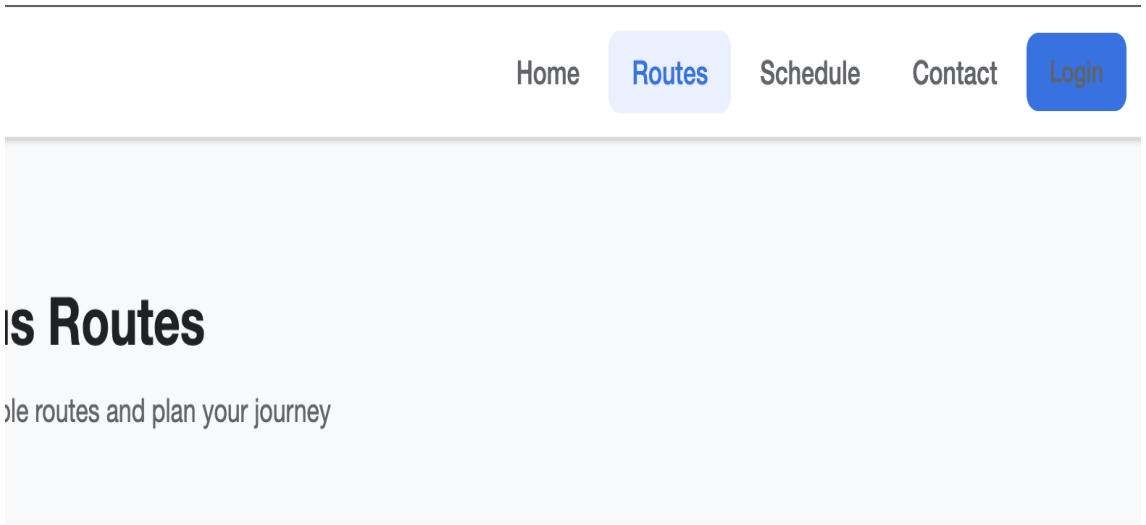
Admin Portal

- admin/dashboard.html
- admin/routes.html
- admin/buses.html
- admin/users.html
- admin/tickets.html
- admin/reports.html
- admin/settings.html

Shared assets: css/style.css • js/app.js • img/ (icons) • webpack configs

Public UI Walkthrough

Public pages (Home → Routes → Schedule → Contact)



- Routes page: cards with route status, frequency, and fare
- Schedule page: filter by route and day; table of trips
- Contact page: form + FAQ accordion (interactive)

Key interactions

- Mobile nav toggle
- Schedule filters + search
- FAQ accordion expand/collapse
- Buttons route to login / register

Demo tip:

Use Routes → pick any route → then go to Login to enter the user/admin portals.

User Journey (Demo Flow)

1. Register
Create an account

→

2. Login
Authenticate (demo)

→

3. Buy Ticket
Search trips + confirm

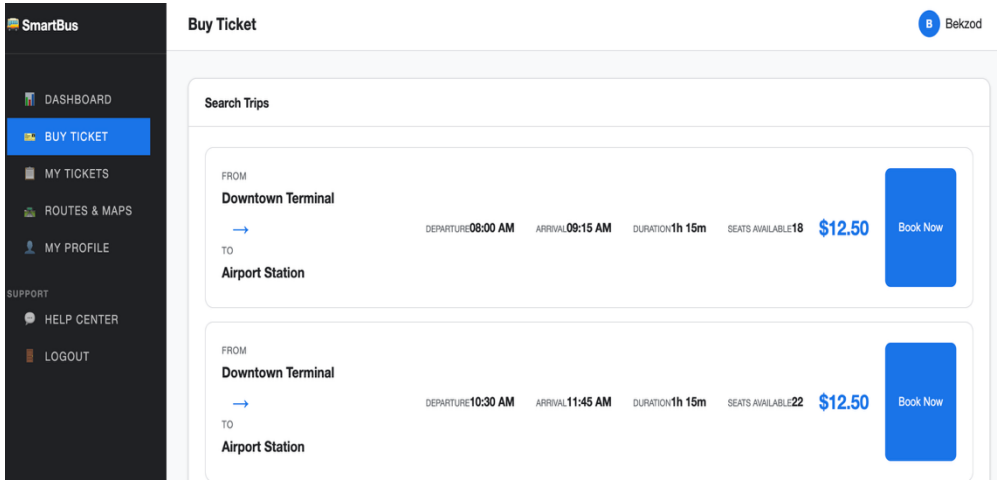
→

4. My Tickets
QR tickets + history

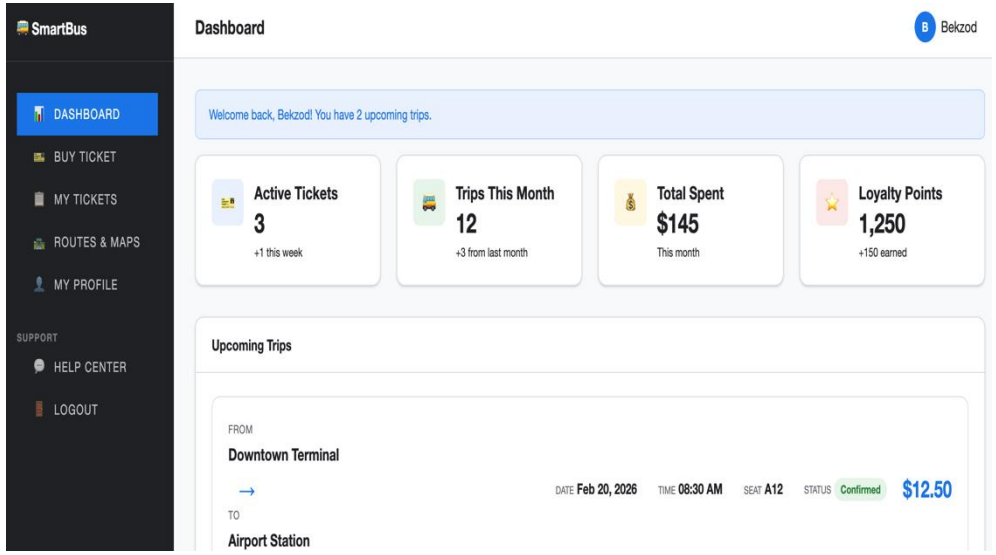
→

5. Dashboard
Upcoming trips + stats

Buy Ticket (mock)

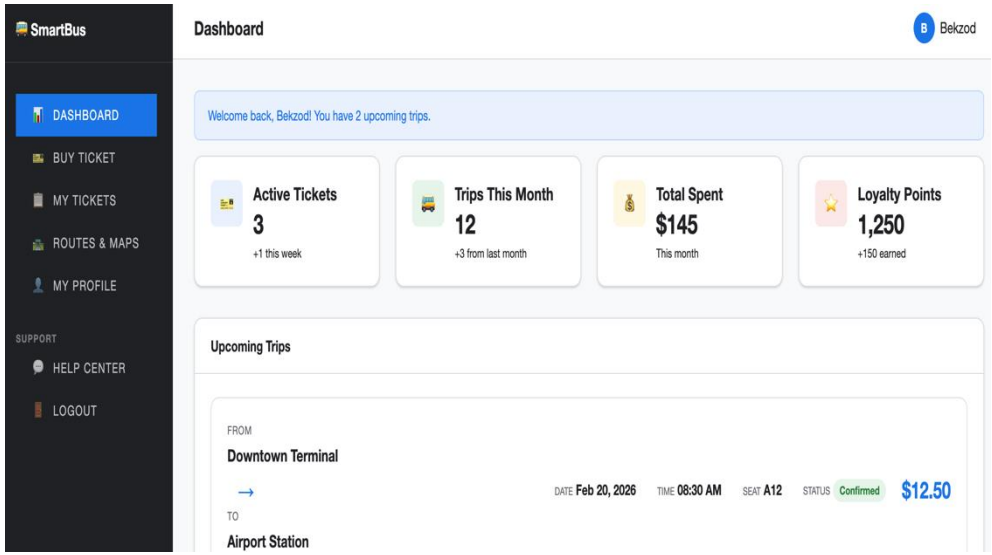


User Dashboard (mock)



User Portal (Key Screens & Features)

Dashboard + My Tickets



- Stats: active tickets, trips, spending, points
- Upcoming trips list with status
- Ticket tabs: Active / Past / Cancelled
- QR code concept per ticket

Buy Ticket

- Trip search (from/to/date/passengers)
- Available trips rendered as cards
- Booking modal: confirm & pay (demo)

Profile + Preferences

- Edit personal info
- Account settings (password)
- Notification preferences + language

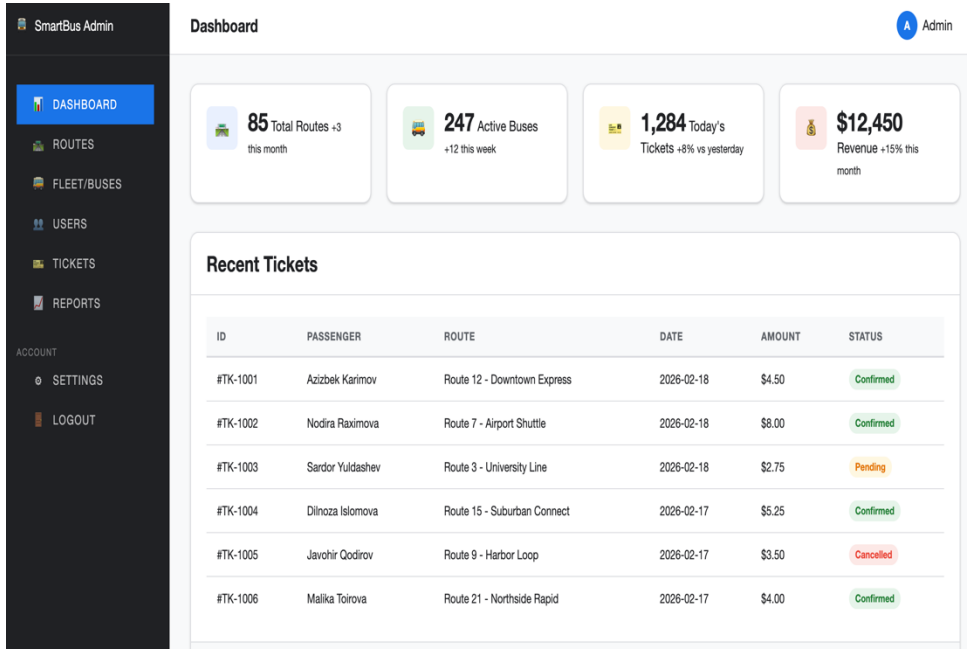
Admin Portal (Operations View)

Admin capabilities (prototype)

- Route management: add/edit routes + status
- Fleet management: buses, assignments, maintenance
- User management: roles (Admin/User/Driver)
- Ticket management: sales KPIs + passes
- Reports & analytics placeholders
- Settings: ticket rules, notifications, security

Login tip: use an email containing “admin” to land on the admin dashboard (demo behavior).

Admin dashboard (mock)



Key Frontend Behaviors (JavaScript)

Reusable UI behaviors (js/app.js)

- Navigation toggle (mobile) and dashboard sidebar toggle
- Modal open/close + overlay click + Escape key handling
- Tab system for switching views (e.g., My Tickets)
- FAQ accordion expand/collapse on Contact page
- Client-side form validation (required fields, email, password match)
- Table search/filtering for lists

Demo login routing

```
function handleLogin(event) {
  event.preventDefault();
  // Demo: redirect based on email
  if (email.value.includes('admin')) {
    window.location.href = 'admin/dashboard.html';
  } else {
    window.location.href = 'user/dashboard.html';
  }
}
```

Note: this repo is a frontend prototype (demo data + UI flows). In a full system, this would call an authentication API and return a token/session.

Technologies & Build Setup

Tech stack

- HTML (multi-page site)
- CSS (shared theme + components)
- Vanilla JavaScript (interactions)
- No backend included (demo flows)

Local dev + bundling (Webpack)

- npm start → webpack-dev-server (opens browser)
- npm run build → production bundle
- Plugins: html-webpack-plugin, copy-webpack-plugin

Quick run commands

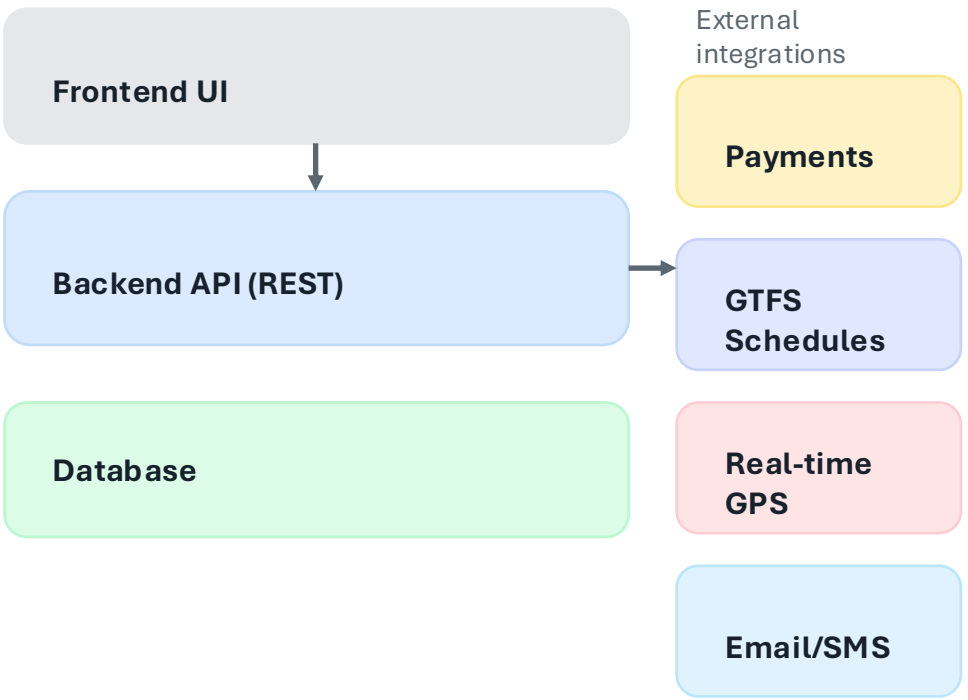
```
npm install
npm start      # local dev server
npm run build  # production build
```

Integration Plan (How this becomes a real system)

Services to integrate

- Authentication & user profiles (JWT / sessions)
- Ticketing API (purchase, cancel/refund, passes)
- Real-time vehicle positions (GPS feed)
- Schedules & routes (GTFS / agency feed)
- Payments (Stripe/PayPal) + receipts
- Notifications (email/SMS for delays and confirmations)
- Admin analytics dashboards

Reference architecture



Live Demo (5–7 minutes)

Suggested click-path

- 1) Home → explain project purpose + key features
- 2) Routes → show route cards (status, frequency, fare)
- 3) Schedule → use filters (Route + Day)
- 4) Contact → FAQ accordion
- 5) Register (optional) → Login
- 6) User portal: Buy Ticket → confirm booking modal → My Tickets → Dashboard
- 7) Admin portal: login with an email containing “admin” → show dashboards and management pages

Demo prep

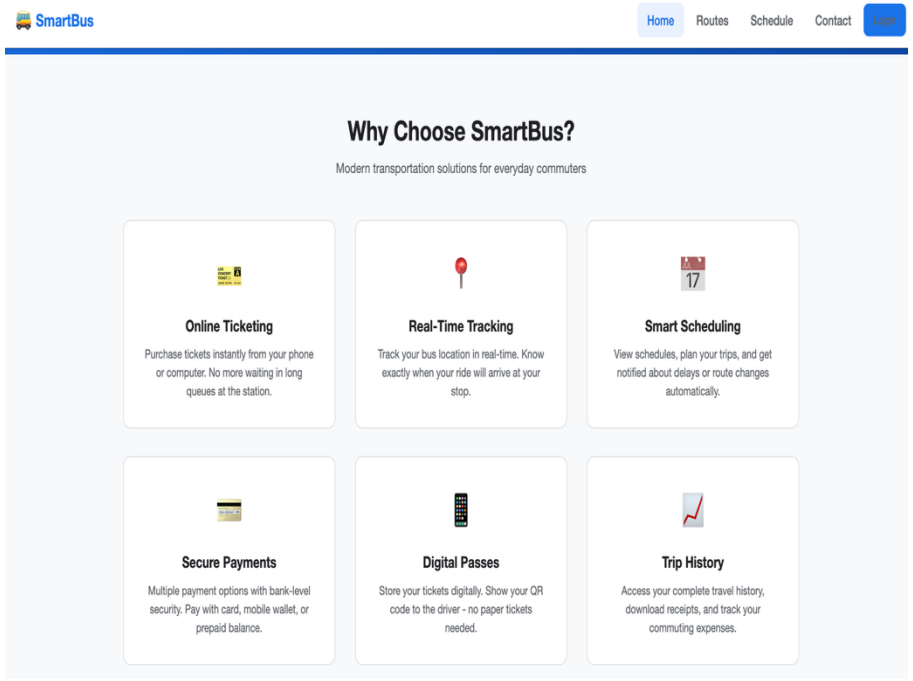
```
Run locally with:
npm install
npm start

Or open the HTML files
directly (static).
```

Future Improvements & Q&A

Next enhancements

- Backend API + database (users, tickets, routes)
- Real-time map view (bus locations)
- Payments + refunds + receipt history
- Role-based access control (Admin / User / Driver)
- Analytics charts on Reports page
- Accessibility + responsive testing
- Automated tests (unit + UI)

Questions?

Thanks!

Repo: https://github.com/sherzodbeek/smart-bus